

ViennaPhysicsEngine

Public classes, simulation walkthrough and game programmer manual



A source-guided book in plain language. Read the first part to understand the objects you use, the second to follow their work, and the third to build a game around them.

Edition and evidence

Prepared 4 October 2026. Physics source: ViennaPhysicsEngine. Clone the main branch for the module-based API described in this guide.

Explanations describe the inspected implementation. Code labeled **pattern** is an integration design; code labeled **example** uses the public API. Snippets were checked against source signatures, but were not compiled for this documentation task.

How to read it

Use the linked contents and PDF bookmarks. Each chapter ends with source IDs. The source map gives the file and line to open in your editor.

Contents

Choose a part, then a chapter.

This book follows the same teaching order as the VVEngine guide: the interface first, the implementation next, and practical use last.

Chapter	Page
Part I - Public classes and API	3
Part II - The implementation step by step	12
Part III - Game programmer manual	23

A useful first reading

Read the class map and lifetime rules before writing code. Then follow the complete update walkthrough. Finish with the starter example, integration chapter and troubleshooting guide.

Source references

A source ID such as S03 is local evidence, not an external citation. The final source map resolves it to a precise file and line in the inspected revision.

What verification means

The book was checked for source consistency, linked navigation and page layout. It does not claim that a new build or a test suite was run.

Part I - Public classes and API

Chapter guide - click a title to jump to its page.

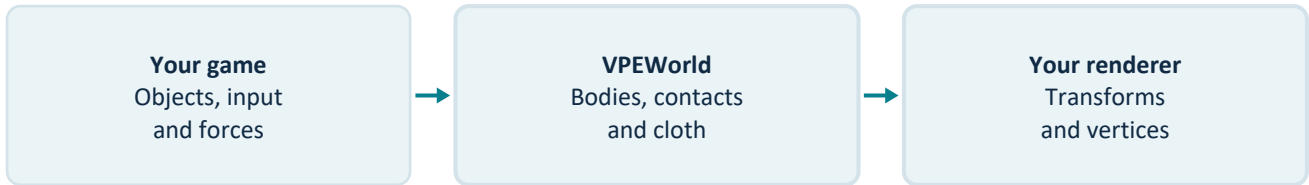
Chapter	Page
1. What the engine gives your game	4
2. The class map	5
3. Numeric types and coordinates	6
4. The world and its stored objects	7
5. Body and Force	8
6. Collision geometry and contacts	9
7. Joint API	10
8. Cloth API	11

Each chapter is a focused reading stop. Follow the order for a first reading; return directly to an API, recipe or troubleshooting page while programming.

1. What the engine gives your game

A CPU simulation world with a direct C++ module API.

ViennaPhysicsEngine computes how rigid objects and cloth move and collide. Your game supplies shapes, starting positions and forces. The world advances them through time. Callbacks let your game copy the result into visible objects.



The entry point is **import VEPHysicsEngine;**. The physics namespace is **vpe**. You construct a **vpe::VPEWorld** directly. There is no backend selection factory, EngineBuilder, renderer initialization or automatic application loop.

Responsibility	Who provides it
Mass, velocity, forces, contacts and joints	ViennaPhysicsEngine
Cloth point motion and mesh positions	ViennaPhysicsEngine
Asset files and mesh/material loading	Your game or VVEngine
Input, gameplay and update scheduling	Your game
GPU buffers, drawing and presentation	VVEngine or another renderer

The useful mental picture

The world stores simulation state. A Body is a moving physical object. A game object is its owner and visible representation. These are related objects, not the same object.

2. The class map

Which types you call, and which types support the simulation.

Type inside VPEWorld	Role / public API use
VPEWorld	Main API: register objects, tick, remove objects and configure simulation.
Body, Force	Game API: rigid state and persistent force entries.
Collider, Polytope	Shape API: convex collision geometry. Collider is an empty base.
Vertex, Edge, Face, signed_edge_t	Shape construction records; maintain their internal links.
Constraint, DistanceConstraint	Joint interface and a center-distance link.
BallSocketJoint, HingeJoint	Joint API: point connection and rotation around an axis.
FixedJoint, SliderJoint	Joint API: fixed relative pose or sliding along an axis.
Cloth	Game API: deformable mesh, transformations and generated vertices.
ClothMassPoint, ClothTriangle, ClothConstraint	Cloth implementation objects; normally created by Cloth.
Contact, ContactTransforms	Solver records and coordinate conversions; normally world-managed.
SatQuery, EdgeQuery, FaceQuery	Detailed collision query records used by narrow phase.
MapWrapper	Vector-backed body registry, despite its map-like interface.

Contact also contains BodyPtr and ContactPoint records. Cloth has private helpers for body poses and accumulated corrections. These explain the implementation; a normal game does not construct them. The API exposes substantial state publicly. Public visibility alone does not make every field a safe gameplay control. Prefer the registration and force methods; update derived data deliberately when editing body fields.

3. Numeric types and coordinates

Know what a position, orientation and mass mean before creating bodies.

The module exports math aliases such as **real**, **glmvec3**, **glmquat** and **glmquat**. These aliases are exported at global scope. The world and its nested classes live in `vpe`. The `_real` literal is available for constants using the configured numeric precision.

Value	Meaning
PositionW, linear_velocityW	World-space position and velocity.
orientationLW	Quaternion taking local orientation into world space; identity is {1,0,0,0} in w,x,y,z order.
Scale	Body geometry scale, included in model and inertia calculations.
mass_inv	Inverse mass: 1/mass for a moving object; zero for a static object.
angular_velocityW	World-space angular velocity; use radians per second.
Force / acceleration	Use one consistent distance, time and mass system across your game.

The ground collision code uses a Y-up plane at $y = 0$. The module also contains coordinate-conversion helpers and a C matrix that swaps axes in the default configuration. The example creates Y-up positions directly. Do not apply a second conversion without tracing your own asset-to-physics path; the helpers are not public game methods.

CMake exposes `VPE_DOUBLE_ACCURACY` and `VPE_FORCE_LEFT_HANDED`, both off by default. The first changes the compiled physics numeric types. The second changes the coordinate setup. Build the module and its consumers consistently; do not mix separately built module interfaces from different configurations.

A practical convention

Start with one world unit as one meter, time in seconds and mass in kilograms. Convert imported visuals once at the game boundary and test a falling cube before connecting a large scene.

4. The world and its stored objects

Registration, identity and ownership are different concerns.

The world registers rigid bodies by their `m_owner` pointer. That pointer connects physics to an object in your game. Physics treats it as an identity; it does not know the C++ type behind `void*`. Give each body a unique, stable, non-null owner address. The built-in ground uses null as its special identity.

Call / member	Actual behavior
<code>addBody(shared_ptr<Body>)</code>	Stores the body, updates grid and matrices. It does not reject duplicate owners.
<code>getBody(owner)</code>	Returns a key/body pair by value. Access <code>.second</code> for the <code>shared_ptr</code> .
<code>m_bodies</code>	MapWrapper stores a vector of owner/body pairs; lookup is linear.
<code>eraseBody(shared_ptr<Body>)</code>	Calls the erase callback and also removes constraints containing that body.
<code>eraseBody(owner)</code>	Removes registry/grid/collider data, but does not perform the same constraint pruning.
<code>clear()</code>	Clears rigid registries and constraints; cloth has separate cleanup.

The world and constraints retain bodies through `shared_ptr`. A `Body` also keeps a raw pointer to its world and a raw pointer to its `Polytope`. Those two objects must remain alive. A `shared_ptr` to a `Body` does not extend the lifetime of its world, shape or game owner.

`MapWrapper::operator[]` returns a pair reference and throws if the key is absent. It does not insert an empty entry like `std::unordered_map`. Because insert appends, duplicate owner keys can create ambiguous lookup and removal.

Removal rule for game code

Keep the body `shared_ptr` on your game object and use that overload when deleting it. Queue deletion during callbacks, then process the queue after tick returns.

5. Body and Force

The objects you use most often.

A Body holds position, orientation, linear and angular velocity, inverse mass, material response and force entries. It also caches matrices, inertia, grid location, previous pose and solver bookkeeping. A static body has inverse mass zero. A dynamic body normally has positive inverse mass.

```
using World = vpe::VPEWorld;
// Constructor parameter order, shown as a guide:
World::Body(worldPtr, name, ownerPtr, polytopePtr,
            scale, positionW, orientationLW,
            linearVelocityW, angularVelocityW,
            inverseMass, restitution, friction);
```

The shorter Body(worldPtr) constructor uses default cube geometry and a static inverse mass. The full constructor supplies defaults for orientation, velocities and response values. It computes local inertia, applies the collision margin factor and updates matrices.

Body API	How to use it
setForce(id, Force), removeForce(id)	A force ID identifies a persistent entry. Setting the same ID replaces it.
m_on_move	Callback receiving residual time and a shared_ptr to the body.
m_on_erase	Callback for detaching the corresponding visible object.
updateMatrices(), inertiaTensorL()	Recompute derived state after deliberate manual changes.
computeModel(position, orientation, scale)	Builds the transform used to place the visible object.

Force separates world acceleration, world force and local force/application-point terms. Acceleration is not multiplied by inverse mass; force is. A local force also generates torque from its local point. The current world-force path does not automatically add a local lever-arm torque.

Gravity is explicit for rigid bodies

The world defines c_gravity, but a new rigid body does not automatically receive a gravity force. The supplied example installs one with Body::setForce.

6. Collision geometry and contacts

A rendered mesh and a collision shape serve different jobs.

Polytope is the rigid collision shape. It stores vertices, edges and faces, their adjacency links and an inertia calculation. The built-in **VPEWorld::g_cube** is a reusable convex cube. A scaled Body can use this same shape without loading a new collision mesh.

The detailed rigid collision test uses the separating axis theorem, or SAT. Its geometry assumptions are convex. An arbitrary concave imported triangle mesh is not automatically a valid Polytope. A game can approximate such an object with convex pieces, but it must create and manage those pieces itself.

Record	What it stores
Vertex	Local position and connected geometry.
Edge / signed_edge_t	Endpoints, adjacent faces and oriented construction references.
Face	Face geometry, normal and boundary links.
Contact	A body pair, current/previous points and persistence information.
ContactPoint	World contact position, lever arms, effective mass, friction and accumulated impulses.
ContactTransforms	Conversions used to compare one body and its geometry with the other.

The world creates Contact objects while detecting collisions. Games normally observe collisions through registered callbacks rather than filling contact points by hand. Solver data is reused between ticks, so deleting bodies or changing shapes during detection can invalidate the assumptions being used.

Shape lifetime

Polytope records contain internal pointers and references. Reserve and finish construction before wiring adjacency; keep the shape stable and alive while any body uses it. Avoid casual copies or vector resizing after links have been established.

7. Joint API

Choose the freedoms that should remain between two bodies.

Class	Connection and remaining freedom
DistanceConstraint	Keeps body centers at a chosen distance; does not create an anchor-based hinge.
BallSocketJoint	Keeps a world anchor coincident; relative rotation stays free.
HingeJoint	Keeps an anchor coincident and leaves rotation about one axis.
FixedJoint	Keeps relative position and orientation fixed.
SliderJoint	Leaves translation along one axis while restricting the other motion.

All derive from Constraint. The world calls **setUp(dt)** to prepare each constraint and **solveVelocity()** during solver iterations. The constraint holds shared_ptrs to its bodies, so keeping a joint alive also keeps those Body objects alive.

```
using W = vpe::VPEWorld;
// Example: bodyA and bodyB are registered shared_ptrs.
auto joint = std::make_shared<W::HingeJoint>(
    bodyA, bodyB, glmvec3{0, 2, 0}, glmvec3{0, 1, 0});
joint->enableLimit(-0.5_real, 0.5_real);
world.addConstraint(joint);
```

A hinge limit uses radians relative to the initial setup. Use $\min < \max$ with a range containing zero. Hinge and slider motors set a target speed with a force/torque bound. Supply a nonzero axis and positive bounds.

Translation, rotation and limit bias controls affect how strongly errors are corrected.

Constraints do not automatically replace collision detection between their connected bodies. Build a mechanism with geometry that can move as intended; do not assume an undocumented collision-filter feature. Remove a joint with `removeConstraint` when the connection should end.

8. Cloth API

A deformable object built from a triangle mesh.

Cloth takes a world pointer, a name, owner identity, move and erase callbacks, vertex positions, triangle indices and fixed-point positions. Optional settings specify bending compliance, substeps and how much applied movement is simulated. Unlike Body, its callbacks are constructor arguments.

Input / method	Meaning
vertices + triangle indices	Original mesh positions and complete, valid triangle connectivity.
fixedPointPositions	Positions matched to mass points; these points are pinned.
bendingCompliance	Larger compliance permits more bending. This is not a direct spring stiffness.
substeps	Smaller cloth time increments inside one world step; must be positive.
generateVertices()	Returns positions in the original mesh vertex order for rendering.
applyTransformation(matrix, simulateMovement)	Moves cloth; simulation mode moves fixed and free points differently.
addCloth / getCloth / eraseCloth / clearCloths	Separate world registry and lifecycle from rigid bodies.

Construction merges coincident positions into physical mass points and preserves a mapping to original vertices. A mesh seam can therefore have several render vertices represented by one simulated point. Cloth cannot be copied or moved because its internal objects refer to each other.

The constructor validates its world, finite geometry, triangle indices, compliance, substep count and movement factor. Degenerate topology and disconnected mass points can be rejected. Cloth registration separately rejects a null cloth and duplicate owner identity.

The render contract

Physics returns positions. Your renderer must update the appropriate mesh and handle any normal, tangent or buffer updates that its material needs. A cloth does not upload to the GPU by itself.

Part II - The implementation step by step

Chapter guide - click a title to jump to its page.

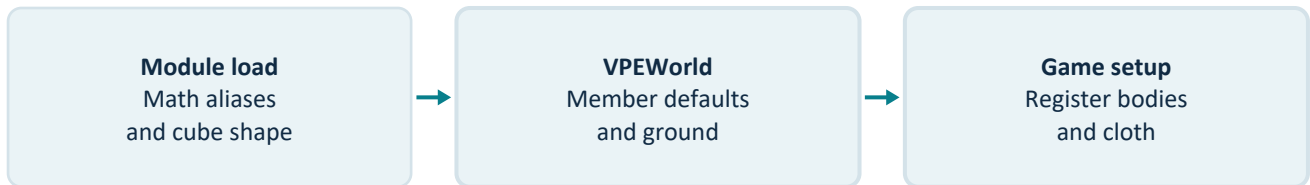
Chapter	Page
9. Files and initialization	13
10. One tick, in execution order	14
11. Time slots, residual time and pause	15
12. How forces become motion	16
13. Broad phase and its spatial grid	17
14. Narrow phase and persistent contacts	18
15. The contact solvers	19
16. Joints inside the solver	20
17. Cloth construction and substeps	21
18. Cloth and rigid bodies share a response	22

Each chapter is a focused reading stop. Follow the order for a first reading; return directly to an API, recipe or troubleshooting page while programming.

9. Files and initialization

What happens when you construct a world.

Most physics implementation is in **modules/VPE.ixx**. It declares the exported module, defines public types and implements the world, rigid bodies, contacts, joints and cloth. The small **modules/VPE.cpp** supplies out-of-line geometry magnitude approximation functions. File extensions do not divide interface and implementation cleanly here: the interface file contains almost all behavior.



VPEWorld has an empty constructor body. Initialization still does work through member initializers: it creates settings, containers, timing state and the special ground body. That ground is static, has a null owner and participates globally in broad phase.

Default	Practical consequence
60 Hz; fixed delta 1/60 second	Physics uses fixed time slots inside tick.
m_loops = 30	Requested maximum contact/joint solver iteration count.
m_solver = 0	All-in-one contact impulse solver is selected.
Grid cell width = 3	Rigid broad phase uses a coarse XZ grid.
Ground collision plane y = 0	Visual terrain is not imported into this collision plane.
No application loop	Nothing advances until the game calls tick.

Construct a stable world, install callbacks and register objects before the first tick. Do not move or copy the world while its bodies refer to its address. Rendering resources can be created independently, but owner identities must remain valid across both systems.

10. One tick, in execution order

Step over this method to see the entire simulation pipeline.

Order	Operation inside a fixed slot
1	Find likely rigid body pairs in broad phase.
2	Create or refresh actual contacts in narrow phase.
3	Warm-start contacts using previous impulse information.
4	Integrate rigid velocities from forces and acceleration.
5	Prepare joints and iterate contact/joint velocity solves.
6	Save previous rigid poses; integrate positions and orientations; update matrices.
7	Integrate every cloth with its own substeps and body contact response.
8	Apply accumulated cloth corrections back to rigid body poses; update changed matrices.

After all required fixed slots, the world updates rigid grid membership. It then invokes rigid move callbacks and cloth move callbacks with the remaining time after the last slot. Finally it records current time as last time. Broad/narrow phase therefore run before rigid velocity integration in each slot. Cloth runs after rigid pose integration and can correct the body result within that same slot. Rendering callbacks come after the complete catch-up loop, not after every individual solver iteration.

Where your systems belong

Gameplay can set forces before tick. Physics computes a new state during tick. Callbacks publish transforms and cloth positions. The renderer then draws that published scene; physics never submits a Vulkan command.

11. Time slots, residual time and pause

The exact clock matters when stepping through frames.

In realtime mode the world sets current time to last time plus the dt you pass. It enters the fixed-step loop while **current_time > next_slot**. This comparison is strict. With the initial next slot at 1/60, a first tick of exactly 1/60 may run callbacks without advancing a fixed physics slot.

```
// Conceptual trace of the inspected clock, not replacement code:
current_time = last_time + dt;
while (current_time > next_slot) {
    simulate_one_slot(m_sim_delta_time);
    last_slot = next_slot;
    next_slot += m_sim_delta_time;
}
publish(current_time - last_slot);
last_time = current_time;
```

A slow render frame can advance several slots in one tick. There is no frame catch-up cap in this loop. A long pause or debugger stop can produce a large amount of work if your game forwards the full elapsed time. Decide whether to clamp elapsed time, discard paused time or deliberately catch up.

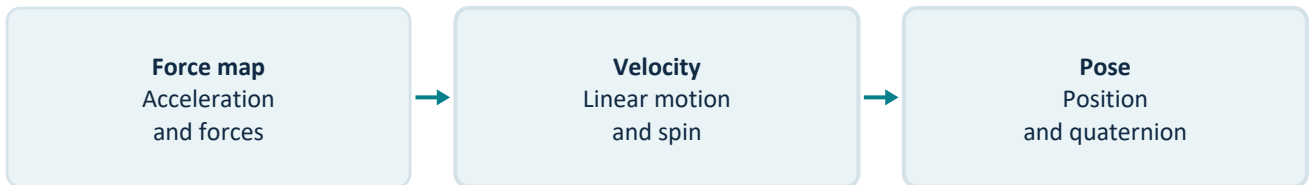
The move callback receives **residual time**, not the full render frame dt. It can estimate a pose between fixed updates. Treat fixed simulation state and the displayed estimate separately. In debug mode tick does not advance current time from dt; the example advances the clock explicitly before calling tick(0).

m_run is present, but tick does not use it as a pause guard. The application must gate ticking. Changing m_sim_frequency alone does not recompute m_sim_delta_time or reset clock slots. Change related values consistently before stepping.

12. How forces become motion

Rigid integration changes velocity first, then pose.

Body::stepVelocity reads persistent force entries. It adds world acceleration directly, adds world and transformed local forces multiplied by inverse mass, and converts local point/force pairs into torque. Inertia converts torque into angular acceleration. The rotational update also accounts for the interaction between angular velocity and the inertia tensor.



A force is persistent until removed or replaced. Installing gravity once makes it act in every later slot. A jump impulse or a one-time hit needs a deliberate policy: a velocity change, an appropriate single-slot force entry, or a game-specific impulse helper. Reapplying a persistent force unintentionally can keep pushing the object forever.

Stored value	Why the world needs it
Local inertia and world inverse inertia	Resistance to rotation, updated for the current orientation.
Previous position/orientation	Pose history used by cloth/body contact handling.
Position and velocity bias	Numerical contact correction, separate from game forces.
Activity/damping bookkeeping	Resting behavior and reduced small motion.
Model, inverse and normal matrices	Geometry transformations needed for collision queries.

Body::stepPosition advances a supplied position and quaternion using current velocities and optional position bias. The helper also touches activity-related state. Even when called with copied poses to estimate a render transform, it is not a completely pure read-only function.

One source of truth

Avoid moving a dynamic visual independently while its Body continues from another pose. Control the physics state and publish the result to rendering.

13. Broad phase and its spatial grid

A fast filter before detailed geometry tests.

The rigid broad phase stores bodies in a sparse two-dimensional grid over X and Z. Each normal body belongs to one cell based on its center. The ground is global and is considered against every body. Neighbor comparisons avoid repeating the same pair, and symmetric owner-pair hashing identifies a contact regardless of pair order.

A grid is cheaper than testing every object against every other object. It does not itself prove collision. Broad phase only prepares pairs; narrow phase then examines their actual shapes. The grid code uses center cells and neighboring cells rather than inserting each shape into every overlapped cell.

Situation	What to consider
Large objects	Two overlapping shapes with distant centers may fall outside neighbor coverage.
Fast motion / large dt	The approach is discrete; do not assume continuous rigid collision detection.
Negative coordinates	Cell conversion uses integer truncation toward zero rather than mathematical floor.
Several catch-up slots	Grid membership is refreshed after the entire tick catch-up loop.
Changed grid width	Existing membership must be rebuilt consistently, not just the scalar changed.

Choose a cell width that suits the body sizes and expected movement in your scene. A larger width reduces missed neighboring candidates but increases the number of detailed tests. Verify boundary cases around cell edges and both sides of zero.

Debugging a missed collision

First check whether the pair entered the contact candidate set. If it did not, investigate grid coverage and owner identity before studying SAT or solver friction.

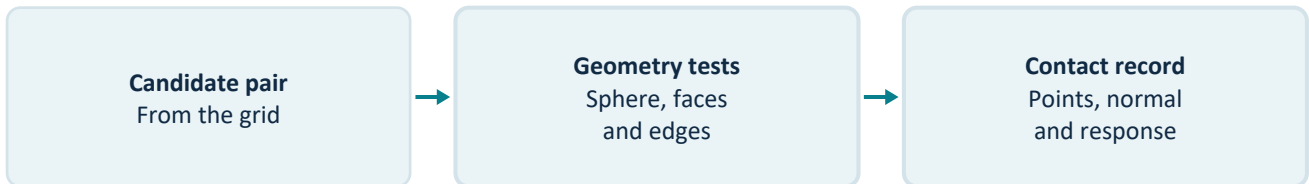
14. Narrow phase and persistent contacts

From a likely pair to contact points.

For a candidate pair the engine first uses a bounding-sphere check to reject clearly separated shapes. Detailed convex testing compares face and edge axes using SAT. If any separating axis exists, the pair does not collide. Otherwise the engine determines a contact direction and penetration information and builds contact points from face/edge geometry.

ContactTransforms moves points, vectors and normals into the spaces needed for these comparisons.

Supporting vertices and face boundaries provide the geometry for clipping and contact generation. The ground follows a special path: body vertices are tested against the $Y=0$ plane.



The world keeps contact records across updates and compares old and new point information. Persistence lets the solver reuse previous impulses instead of starting from zero each time. Narrow phase also removes stale contacts when a pair is no longer present or valid.

A registered collision callback is invoked through this detection path. It is not a full begin/stay/end event abstraction. Persistent-contact reuse can change when detection code runs. If your gameplay needs distinct events, maintain your own pair state and verify it against the observed behavior.

Do not mutate the current traversal

A collision callback may run while the world is iterating contact or registry data. Record an event or queue a deletion. Process structural changes after tick returns.

15. The contact solvers

Impulses are short velocity corrections at contact points.

A contact impulse changes linear velocity and also angular velocity through the lever arm from the body center. The solver considers both bodies. Effective mass describes how strongly the two bodies respond at that particular contact, including their resistance to rotation.

Setting / solver	Behavior
<code>m_solver = 0</code>	Solves contact response using an all-in-one three-dimensional matrix formulation.
<code>m_solver = 1</code>	Solves the normal direction separately from two tangent directions.
Normal impulse	Accumulated amount is clamped nonnegative so the contact pushes rather than attracts.
Friction impulse	Tangent magnitude is limited by the normal impulse times friction.
Restitution	Controls bounce for the contact response.
Bias / slop	Corrects penetration while allowing a small tolerance.

Warm starting applies useful previous impulse information to persistent resting contacts. This reduces the correction work needed for stacks. `m_use_warmstart` is enabled by default; the separate single-contact warm-start option is off and the source comments note stability concerns.

`calculateImpulses` iterates contact response and every joint velocity solve. `m_loops` defaults to 30. In realtime mode the routine also uses a wall-clock budget associated with the fixed slot. Debug mode omits that time-budget stop. Use positive loop counts; the loop is written to execute at least once.

A fixed time step therefore does not guarantee an identical number of iterations on every machine. Wall-clock stopping, container order and initial conditions can affect results. Do not advertise deterministic multiplayer physics without a separate reproducibility evaluation.

16. Joints inside the solver

A connection solves velocity error repeatedly.

Before iterations, `setUp` computes joint geometry, effective masses and bias for the current poses. Each `solveVelocity` call applies impulses that reduce unwanted relative motion. The same joint is solved repeatedly alongside contacts, so a mechanism and the surfaces it touches influence each other.

Joint	Error it tries to reduce
DistanceConstraint	Difference between the current center distance and target distance.
BallSocketJoint	Separation between the two local representations of one world anchor.
HingeJoint	Anchor separation and rotation away from the hinge axis; optional angle limits and motor.
FixedJoint	Both anchor separation and relative orientation error.
SliderJoint	Translation perpendicular to its axis and relative rotation; optional travel limit and motor.

The constructor converts a supplied world anchor into each body's local coordinates. As the bodies move, their anchors transform back to world space. A hinge leaves the intended rotation free; a slider leaves the intended axis translation free. Limits add a constraint only when motion reaches the relevant bound.

Static bodies require special angular handling because a zero inverse mass must not let a fixed support rotate under a joint impulse. Bias settings trade correction strength against jitter. Excessive correction, very different body masses, conflicting contacts or too few iterations can make a mechanism unstable.

A debugger sequence

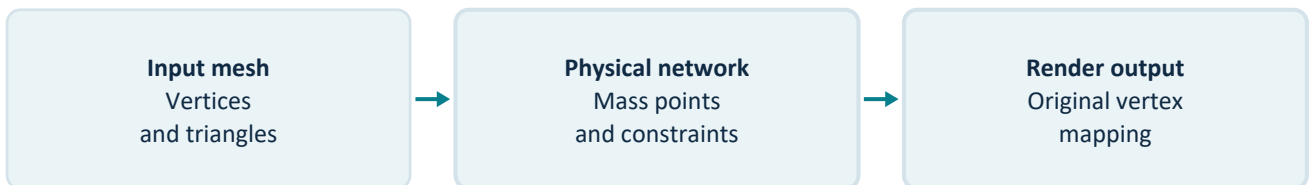
Break in a joint constructor, then `setUp`, then `solveVelocity`. Watch the local anchors stay constant while world anchors, errors and accumulated impulses change.

17. Cloth construction and substeps

The render mesh becomes a connected network of points.

Cloth first validates the mesh. It merges matching vertex positions into physical mass points and records which original render vertices refer to each point. Triangle topology identifies neighbors. Constraints connect triangle edges; bending connections relate points across shared edges. Triangle area contributes to point mass.

Fixed-point positions identify pinned points. The cloth constructor applies a small initial rotation to avoid a perfectly planar starting arrangement having no variation in one direction. Keep that behavior in mind when comparing input vertices with the very first generated output.



During integrate, each world slot is divided into the configured cloth substeps. Each substep applies damping and external acceleration, predicts motion, solves compliant constraints and handles rigid-body and ground contacts. Cloth receives gravity automatically through its integration path, unlike the rigid-body force registration policy.

The compliant solver includes the substep duration in its response. Compliance divided by time-step squared contributes to the solve; it is not interchangeable with a spring stiffness number. More substeps cost more CPU work and can improve contact and constraint handling.

Preserve the topology mapping

Use `generateVertices` for the visible mesh. Do not publish the merged physical point array as though it had the original render vertex count or ordering.

18. Cloth and rigid bodies share a response

The rigid body reacts within the same world slot.

Cloth contact tests use nearby rigid bodies and interpolate their previous and current poses across cloth substeps. They can inspect swept surface motion while resolving point/body interactions. This is a cloth-specific contact path; it is not a general guarantee of continuous collision detection for all rigid objects.

A contact shares positional correction according to the participating masses. Normal and friction impulses affect cloth point velocity and the body's linear/angular response. Angular response uses the contact lever arm. Cloth also records corrections for the part of a substep remaining after a contact.

During cloth integration	After cloth integration in tick
Accumulate body contact response	Apply remaining-time linear/angular pose corrections.
Accumulate positional bias contributions	Apply combined position-bias corrections.
Use previous/current rigid poses	Update body matrices if the corrected pose changed.
Solve cloth constraints and contacts	Clear correction buffers before the next slot.

This ordering lets a cloth push or pull a dynamic body without waiting for another rendered frame to apply every positional effect. It also means you should inspect the body after the cloth stage when tracing the final fixed-slot pose.

`applyTransformation` is a cloth-specific movement API. With `simulateMovement` false the transformation moves the cloth directly. With true, fixed and free points receive different proportions using the movement simulation factor, so prescribed motion can deform the cloth. It is applied to current points, not a renderer-only model transform.

Cleanup is independent

The world has a separate cloth registry and `clearCloths`. Calling `clear` for rigid bodies alone does not remove cloth objects or invoke their erase callbacks.

Part III - Game programmer manual

Chapter guide - click a title to jump to its page.

Chapter	Page
19. Clone, build and import the engine	24
20. A minimal falling body	25
21. Store game objects without dangling owners	26
22. Connect the renderer and load assets	27
23. Put physics in the game loop	28
24. Forces, jumps and direct edits	29
25. Collision events and object deletion	30
26. Build a mechanism	31
27. Add a cloth object	32
28. Tune performance and stability	33
29. Troubleshooting by symptom	34
30. Step through the supplied example	35
31. Reference, glossary and next steps	36

Each chapter is a focused reading stop. Follow the order for a first reading; return directly to an API, recipe or troubleshooting page while programming.

19. Clone, build and import the engine

Use the main branch, as with VVEngine.

Install Git, Visual Studio with its C++ workload and CMake tools, and CMake 3.28 or newer. Clone the repository normally. From PowerShell, build the standalone library and tests:

```
cd C:\data\GitHub
git clone https://github.com/hlavacs/ViennaPhysicsEngine.git
cd ViennaPhysicsEngine
.\build_cmake.cmd debug --without-vve --tests
```

Use **import VEPhysicsEngine;** in your C++ program and link **ViennaPhysicsEngine::ViennaPhysicsEngine** through CMake. An import statement alone does not build the module. The project requires C++23 module support and GLM. Its base CMake minimum is 3.28. Building the VVEngine example in-tree has additional import std/compiler/CMake constraints, including a Ninja generator requirement on Windows.

CMake option	Purpose / default
VPE_BUILD_EXAMPLES	VVEngine-rendered example; ON.
VPE_BUILD_TESTS	Standalone physics checks; OFF.
VPE_VVE_IN_TREE	Build sibling VVEngine; defaults ON on Windows.
VPE_DOUBLE_ACCURACY	Double-precision physics math; OFF.
VPE_FORCE_LEFT_HANDED	Alternate coordinate setup; OFF.

Current sibling compatibility

The example source used for this guide imports the VEEngine module name. Your current renderer project uses VVEngine. Adapt that bridge to the current facade and check target compatibility before building the visual example.

20. A minimal falling body

Create a cube, install gravity, then advance time.

```
#include <memory>
import VEPHysicsEngine;

struct GameObject { int id = 1; };

int main() {
    GameObject owner;           // Stable address.
    vpe::VPEWorld world;        // Outlives the body.
    using W = vpe::VPEWorld;
    auto body = std::make_shared<W::Body>(
        &world, "falling cube", &owner, &W::g_cube,
        glmvec3{1,1,1}, glmvec3{0,4,0},
        glmquat{1,0,0,0}, glmvec3{0,0,0},
        glmvec3{0,0,0}, 1.0_real, 0.2_real, 0.8_real);
    body->setForce(0ul,
        W::Force{{0, world.c_gravity, 0}});
    world.addBody(body);
    for (int frame = 0; frame < 120; ++frame)
        world.tick(1.0 / 60.0);
    world.eraseBody(body);
}
```

This is an API example, not a complete visible game. It needs the physics module target and its dependencies. Inverse mass 1 means a mass of one in the chosen unit system. The cube starts above the built-in ground and falls because force entry zero supplies world gravity acceleration.

The owner lives on the stack at a stable address. No rendering callback is installed, so no visual is produced. The world still stores and advances the body. Cleanup uses the `shared_ptr` overload so future joints would be pruned correctly.

First debugger watch list

Watch `m_positionW`, `m_linear_velocityW`, `m_mass_inv` and `m_forces` on the body, then `loop/current_time/next_slot` on the world. Expect the strict time-slot comparison described earlier.

21. Store game objects without dangling owners

Design lifetime before adding collision callbacks.

Store a game object at an address that remains stable while physics uses it. A vector of `GameObject` values can relocate its elements during growth. A vector of `unique_ptr<GameObject>` keeps each pointed-to object stable while the container moves the pointer values.

Object	Required lifetime
VPEWorld	Outlives every Body/Cloth still using its raw world pointer.
Polytope	Outlives every Body that borrows it.
Game owner	Lives until the body/cloth is unregistered and callbacks stop using it.
Rigid body shared_ptr	May be held by the world, game object, contacts and joints.
Visible scene object / mesh	Lives while move callbacks publish into it.
Callbacks and their captured objects	Captured renderer, scene and game state remain valid until detached.

`shared_ptr` solves only the Body lifetime. It does not make a `void*` owner safe, keep a borrowed shape alive or keep a renderer capture alive. A robust `GameObject` stores its body handle and visible handles together, while a scene controller owns the world and shared shapes.

Use an explicit shutdown sequence: stop ticking; clear cloth while its visible resources still exist; clear rigid bodies and joints while erase callbacks still have access to the game/renderer; release any retained contacts or external handles deliberately; then destroy scene, shapes and world in a safe order.

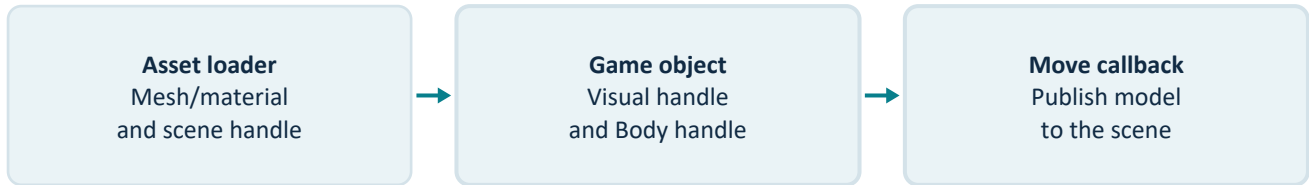
A subtle clear detail

`clear` does not reset all world data. In particular cloth is separate, and contact/time/ground state is not a complete fresh-world reset. Destroy and recreate the world when you need a clean simulation rather than assuming `clear` resets everything.

22. Connect the renderer and load assets

Your bridge turns physics results into visible state.

Load visual meshes, materials and textures through VVEngine or your asset system. Create a collision shape separately. A cube mesh can pair with `g_cube`; a complex model usually needs a deliberately chosen convex approximation. Loading a model file does not automatically register a VPE body.



```
// Pattern: publishModel is your renderer adapter.
body->m_on_move = [publishModel](
    double residual, std::shared_ptr<W::Body> b) {
    auto p = b->m_positionW;
    auto q = b->m_orientationLW;
    // Publish fixed pose; no extrapolation is required.
    publishModel(W::Body::computeModel(p, q, b->m_scale));
};
// Bind the visible-object cleanup in m_on_erase too.
```

The callback signature is real; `publishModel` is an application adapter, not a VPE method. Fixed-pose publishing is the easiest first integration. If you add a residual-time estimate, do not overwrite the stored physics pose. The example uses copied poses with `stepPosition`; remember that helper can still modify activity bookkeeping. For cloth, publish `generateVertices` into the appropriate deformable mesh update API. Recompute normals/tangents if required. Your renderer handles CPU-to-GPU copying, synchronization and draw calls. VPE never acquires a graphics device, image, descriptor or command buffer.

Visible ground versus collision ground

A textured plane in VVEngine can display the floor. VPE's built-in ground collision is still the $Y=0$ plane. Keep them aligned; a textured terrain mesh does not silently change collision height.

23. Put physics in the game loop

Control when gameplay writes and rendering reads.

```
// Pattern: one application update, elapsed time in seconds.
pollInput();
updateGameplay(elapsed);
applyQueuedPhysicsCommands();
if (!paused)
    world.tick(elapsed);
processQueuedCollisionEvents();
processQueuedObjectRemoval();
updateCameraAndUI(elapsed);
drawScene();
```

These are application functions, not VPE methods. The order matters: gameplay can set this update's forces before physics; callbacks publish results during tick; structural edits happen after tick; rendering sees the published scene.

If VVEngine already owns the loop through a user system, put the physics call in that system's update rather than opening a second independent loop. Use the elapsed time that VVEngine provides, in seconds. Keep the renderer alive for callbacks and create bodies only after their visible owner objects have stable identities.

Game action	Loop policy
Pause / resume	Skip tick while paused; do not forward accumulated paused time on resume.
Debugger step	Use a controlled delta or debug clock mode.
Slow frame	Choose an explicit clamp/catch-up policy for incoming elapsed time.
Physics frequency change	Adjust delta and clock state together before resuming.
Multithreading	Serialize world mutation and ticking; no thread-safe world API is promised.

Measure before changing settings

Track frame time, number of fixed slots per tick, solver time and body/cloth counts. A slow frame can cause more physics catch-up and make the following frame slower too.

24. Forces, jumps and direct edits

Use a clear policy for continuous and one-time motion.

Continuous thrust or gravity belongs in a persistent Force entry. Choose force IDs by purpose, for example gravity, player thrust and wind, so one subsystem does not overwrite another's entry. Release an input by removing its force or replacing it with zero according to your policy.

```
// Example: replace one persistent acceleration entry.
W::Force thrust;
thrust.m_accelW = glmvec3{3, 0, 0};
body->setForce(1ul, std::move(thrust));
// Later, when thrust stops:
body->removeForce(1ul);
```

For a one-time jump or hit, do not leave a continuous force active indefinitely. The engine exposes body velocity fields, so a game can implement an instantaneous velocity change with a chosen mass/impulse policy. Verify contact/resting behavior after doing so; a public field write does not execute a separate high-level wake-up API.

Manual edit	Derived state to inspect
Position or orientation	Update matrices, previous-pose assumptions and grid membership.
Scale or inverse mass	Recompute local inertia and derived world matrices/inertia.
Teleport across cells	Move body in the grid as well as updating its pose.
Replace shape	Keep old/new shape lifetimes valid; refresh geometry-derived data.
Large pose changes	Persistent contact data may no longer match the previous pose.

For complex reset or respawn, removal and reconstruction is often easier to reason about than editing many internal fields. Perform it outside tick and reattach required forces, callbacks and joints.

25. Collision events and object deletion

Convert detection callbacks into stable gameplay events.

Register a collider callback with `addCollider(body, callback)`. The callback receives two `shared_ptr<Body>` values. Their `m_owner` pointers let your game recover the corresponding objects using its own type and identity rules. The ground may have a null owner, so handle that explicitly.

```
// Pattern: record an event; consume it after tick.
world.addCollider(body,
    [&events](std::shared_ptr<W::Body> a,
              std::shared_ptr<W::Body> b) {
        events.push_back({a, b});
    });
// After tick: inspect owners, apply gameplay effects,
// then erase selected bodies via eraseBody(shared_ptr).
```

`events` is an application container with a compatible record type. Retaining `shared_ptr`s in that queue keeps the bodies alive; it does not keep their owners alive. Drain or invalidate events before deleting game objects.

A gameplay system should deduplicate events when a contact may be observed repeatedly. If it needs begin/end semantics, explicitly track body pairs and decide how to reconcile them with detection behavior. Do not treat every callback as a fresh collision or promise one callback per rendered frame.

Deletion step	Purpose
Record request in the callback	Avoid altering containers under active traversal.
Finish tick	Let contact and cloth processing complete.
Unregister using stored body handle	Also remove connected constraints through that overload.
Detach visual and owner state	Keep erase callback captures valid until it finishes.
Discard queued references/events	Prevent later gameplay access to deleted owner pointers.

Sources: S05, S08 - see source map

26. Build a mechanism

Anchors and axes should be checked visually.

Start with two bodies and one joint. Use a static support with inverse mass zero and a dynamic moving piece.

Set the joint anchor in world coordinates at the intended connection, and give hinge/slider joints a nonzero axis with the intended direction.

```
// Example: a hinge motor between registered bodies.
auto hinge = std::make_shared<W::HingeJoint>(
    support, door, hingePoint, glmvec3{0,1,0});
hinge->enableLimit(-1.0_real, 1.0_real);
hinge->enableMotor(0.5_real, 2.0_real);
world.addConstraint(hinge);
// Stop the motor without deleting the connection:
hinge->disableMotor();
// Remove the connection when the door breaks off:
world.removeConstraint(hinge);
```

The speed is angular for a hinge and linear for a slider. The motor bound is a solver control, not a promise of a fully modeled physical actuator. Check the source comments for the particular joint's per-iteration interpretation before assigning real equipment parameters.

Draw debug points at both reconstructed anchors and a line along the axis. Watch whether the joint's intended motion is clear before adding many bodies. Build chains and ragdolls incrementally. Very different masses, incompatible limits and geometry that blocks its own motion can create large constraint errors.

When it jitters

Check anchor placement, axis direction, initial overlap, mass ratio, timestep and solver passes before raising bias. Stronger bias can hide an error temporarily while increasing instability.

27. Add a cloth object

Keep simulation and visual mesh identities connected.

```
// Pattern: input data comes from your mesh loader.
auto cloth = std::make_shared<W::Cloth>(
    &world, "banner", stableOwner,
    onClothMove, onClothErase,
    vertices, triangleIndices, pinnedPositions,
    0.1_real, 8, 0.8_real);
world.addCloth(cloth);
// In onClothMove(residual, clothPtr):
//   publishPositions(clothPtr->generateVertices());
// At removal:
world.eraseCloth(stableOwner);
```

vertices is vector<glmvec3>; indices are vector<uint32_t>; pinnedPositions is vector<glmvec3>. The move callback receives double and shared_ptr<Cloth>; the erase callback receives shared_ptr<Cloth>. Adapter names in this pattern are supplied by your game.

Pin positions must match intended physical point positions. Coincident mesh positions are merged, so verify seams and duplicated render vertices behave as expected. Supply complete triangles and finite values. Handle constructor validation failure at asset-loading time rather than halfway through a frame.

Control	Effect to test
Substeps	Contact reliability versus CPU cost.
Stretch / bending compliance	How much the mesh deforms under its constraints.
Air / quadratic damping	How quickly motion and fast motion are reduced.
Constraint damping	Reduction of relative constraint motion.
Friction / restitution	Sliding and bounce at contacts.
Movement simulation factor	How free points follow prescribed cloth transformations.

Begin with a small mesh and pinned top edge. Confirm gravity, vertex mapping and cleanup first; add body contact and prescribed movement afterwards. No cloth self-collision behavior is assumed by this guide.

28. Tune performance and stability

Change one measured variable at a time.

Setting / scene change	Likely trade-off
Higher world frequency	Smaller rigid steps; more slots and CPU work per second.
More solver iterations	Potentially stronger contact/joint convergence; extra work.
Realtime solver budget	May stop iterations sooner on expensive scenes.
Larger grid cells	Broader candidate coverage; more candidate pairs.
More cloth substeps	More frequent cloth/contact solves; higher CPU cost.
Dense cloth mesh	More points, constraints and contact tests.
Warm starting	Reuse contact response; inspect stability of resting stacks.

Measure typical gameplay and worst cases. A benchmark containing only free-falling bodies does not represent a stack, a joint chain or cloth draped over moving bodies. Count candidates and contacts as well as body count; expensive overlap can dominate a relatively small scene.

Avoid treating the public constants as independent sliders. Frequency and delta, mass and inertia, pose and matrices/grid, cloth compliance and substep duration all interact. Record the configuration with saved scenarios so a regression can be reproduced.

For replay or multiplayer, verify repeatability across runs and machines before relying on identical outcomes. This implementation includes a realtime wall-clock solve budget and container traversal effects. A fixed delta alone is insufficient evidence of determinism.

A small regression scene set

Keep a falling cube, a resting stack, a friction slope approximation, a hinge/slider mechanism and a cloth/body contact scene. Record pose, velocity and constraint-error trends, not just whether the application stayed open.

29. Troubleshooting by symptom

Find the earliest stage where the state becomes wrong.

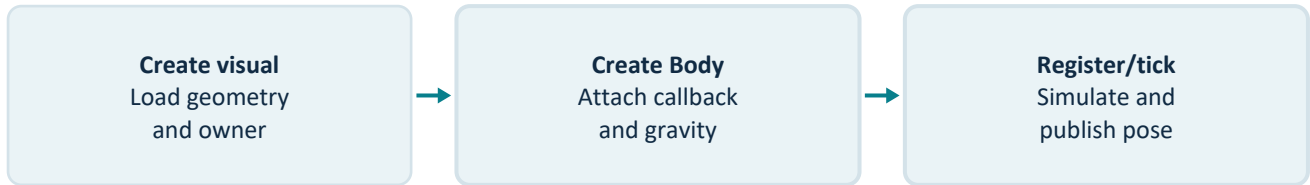
Symptom	First things to inspect
Body does not fall	Inverse mass positive? Gravity force installed? tick advancing slots?
Visible object stays still	Move callback installed? Correct visible handle and transform publication?
Body passes through another	Grid candidate coverage, dt/velocity, convex shape validity and narrow phase.
Ground and visual floor disagree	Physics ground is $y=0$; align the visual plane and object units.
Joints stretch or jitter	Anchor/axis, overlap, mass ratio, timestep, bias and solver budget.
Cloth constructor throws	Finite vertices, index bounds, complete triangles, topology and positive substeps.
Cloth tears visually at seams	Publish original-order generateVertices and verify mesh index/vertex mapping.
Crash after deletion	Dangling owner/shape/callback capture; owner overload leaves joint references.
Long freeze after pause	Large dt creates unlimited fixed-slot catch-up.
Picking behaves oddly	pickBody ranks directional alignment; it is not a geometric raycast.

pickBody also assumes there is a body candidate; guard empty body storage before calling it. For precise ray hits, provide a separate geometric picking implementation and convert the chosen result to a body identity. Log the state before and after the failing stage. If a pair never enters broad phase, changing friction cannot repair it. If physics moves correctly but rendering is stale, changing solver passes cannot repair the visual bridge.

30. Step through the supplied example

Follow one object from input mesh to screen.

Start at world and visible-engine creation. Find the function that constructs a Body. It creates a shared_ptr with the world, name, game owner, cube shape, scale, initial pose and material response. It then assigns move/erase callbacks and installs the gravity force when required.



Next break at addBody and inspect m_bodies and the grid. At tick, distinguish a render update from a fixed slot. Step into broadPhase, narrowPhase and calculateImpulses only for the body you are investigating. Watch its velocity change, then its pose and matrices change.

At the move callback, watch the owner identity lead back to the visible object. The callback computes or estimates a model and sends it through the rendering bridge. GPU activity happens later in the visual engine. The physics library stops at CPU data and callback execution.

For cloth, follow the example's make_shared<Cloth> path, then addCloth. Step into integrate and compare generated positions with the loaded render mesh order. After body contact, inspect both the point velocity and the body correction buffers before tick applies them.

Historical example versus your renderer

The source still uses VEngine in its import. Translate the renderer-facing calls to the current VVEngine facade when integrating your checkout. The physics module name remains VEPHysicsEngine.

31. Reference, glossary and next steps

A compact map to the API and the source.

Task	Use
Create the world	<code>vpe::VPEWorld; import VEPysicsEngine.</code>
Create a rigid body	<code>VPEWorld::Body</code> + a stable Polytope and owner.
Register / remove rigid objects	<code>addBody; eraseBody(shared_ptr).</code>
Apply continuous input	<code>Body::setForce / removeForce.</code>
Observe / publish	<code>addCollider; Body::m_on_move / m_on_erase.</code>
Connect bodies	<code>Joint shared_ptr + addConstraint / removeConstraint.</code>
Create deformable geometry	<code>Cloth + addCloth; generateVertices</code> for the renderer.
Advance / clean up	<code>tick(seconds); clearCloths</code> and rigid cleanup separately.

Broad phase: cheap pair filtering. **Narrow phase:** actual geometry/contact detection. **Impulse:** an instantaneous velocity correction. **Inertia:** resistance to rotation. **Constraint:** a rule restricting relative motion. **Compliance:** permitted softness. **Warm start:** reuse prior contact impulses. **Residual time:** elapsed time after the last fixed slot.

Use the source map on the following pages while debugging. The standalone test translation unit contains API and cloth/body regression scenarios, but this documentation task did not run those checks. Start integration with the minimal body and add rendering, events, joints and cloth one at a time.

What a complete game integration contains

Stable identities and lifetimes, valid convex shapes, a controlled loop, a render bridge, queued structural edits, explicit cleanup and measured regression scenes.

Source map

Open these files in the engine repository.

File paths below are relative to the repository root.

Physics source: ViennaPhysicsEngine. Clone the main branch for the module-based API described in this guide.

ID	File and line	What it explains
S01	modules/VPE.ixx:98	Module, numeric types and coordinate conventions
S02	modules/VPE.ixx:237	World and public state
S03	modules/VPE.ixx:268	Forces and convex geometry records
S04	modules/VPE.ixx:461	Rigid-body data, construction and integration
S05	modules/VPE.ixx:954	Body storage, registration and removal
S06	modules/VPE.ixx:1234	Complete world update order

Line numbers are one-based. Search the named class or method if the repository has changed after this edition. Each map lists only the sources for this book.

Source map - continued

Open these files in the engine repository.

File paths below are relative to the repository root.

Physics source: ViennaPhysicsEngine. Clone the main branch for the module-based API described in this guide.

ID	File and line	What it explains
S07	modules/VPE.ixx:1361	Spatial grid and contact candidates
S08	modules/VPE.ixx:1377	Contact detection, persistence and callbacks
S09	modules/VPE.ixx:1573	Solver iterations and time budget
S10	modules/VPE.ixx:1596	SAT queries and detailed convex collision tests
S11	modules/VPE.ixx:1869	Joint interface and distance constraints
S12	modules/VPE.ixx:2082	Hinge, fixed and slider joints

Line numbers are one-based. Search the named class or method if the repository has changed after this edition. Each map lists only the sources for this book.

Source map - continued

Open these files in the engine repository.

File paths below are relative to the repository root.

Physics source: ViennaPhysicsEngine. Clone the main branch for the module-based API described in this guide.

ID	File and line	What it explains
S13	modules/VPE.ixx:2948	Cloth points and collision response
S14	modules/VPE.ixx:3206	Cloth constraint solving
S15	modules/VPE.ixx:3340	Cloth construction, integration and publication
S16	modules/VPE.ixx:3884	Cloth topology construction
S17	examples/physicsexample/physicsexample.cpp:310	Example body and gravity setup
S18	examples/physicsexample/physicsexample.cpp:265	Example cloth and rendering bridge

Line numbers are one-based. Search the named class or method if the repository has changed after this edition. Each map lists only the sources for this book.

Source map - continued

Open these files in the engine repository.

File paths below are relative to the repository root.

Physics source: ViennaPhysicsEngine. Clone the main branch for the module-based API described in this guide.

ID	File and line	What it explains
S19	CMakeLists.txt:3	Build options and module target
S20	tests/vpe_compile_check.cpp:13	Standalone API and physics regression checks
S21	modules/VPE.cpp:6	Out-of-line geometry implementation

Line numbers are one-based. Search the named class or method if the repository has changed after this edition. Each map lists only the sources for this book.